

Troom

Farm & Agribusiness Management System

System Architecture & Module Map

Laravel 13 / PHP 8.3 monolith

Generated 2026-07-23

Contents

- 1. System Overview
- 2. Roles & Access Control
- 3. Module Map
- 4. Core Data Model & Relationships
- 5. Cross-Cutting Flows
- 6. External Integrations

1. System Overview

Troom is a single Laravel 13 (PHP 8.3) monolith that runs the operations of a horticulture farm business, from land preparation through crop production, harvest, packhouse traceability, sales, and finance — plus a secondary stables (horse-riding) revenue line. It is a server-rendered application (Blade views, no separate SPA/API frontend) with one small stateless API surface for external webhooks and a scale-reading feed.

- **Structure:** standard Laravel MVC — `app/Http/Controllers` (49 controllers), `app/Models` (~70 Eloquent models), `app/Services` for external integrations, `app/Support` for cross-cutting business logic (access control, posting rules, reference data).
- **Routing:** `routes/web.php` — session-authenticated, CSRF-protected, organized into 18 numbered "modules" mirroring the business spec. `routes/api.php` — stateless, token/signature-verified, used only for the M-Pesa Daraja callbacks and the digital weigh-scale feed.
- **Authorization:** single `role` column on `User`, enforced by a `CheckRole` middleware plus a central `ModuleAccess` map (see §2) — there is no separate permissions table.
- **Background work:** queued jobs for AI report generation and KPI narrative generation (`app/Jobs`); an `ActivityObserver` model observer writes an audit trail for all model changes.

2. Roles & Access Control

Every user has exactly one `role`. `owner` is a super-admin and bypasses all module checks. All other roles are granted write access to specific modules via `App\Support\ModuleAccess::MAP` — the single source of truth for who can manage what. Master Data and Crop Cycles are readable by every authenticated role; only writes are gated.

Role	Label	Typical focus
owner	Admin	Super-admin, unrestricted
md	Managing Director	Executive: finance, analytics, crop cycles, WhatsApp ops
horticulture_manager	Horticulture Manager	Broadest operational role: master data, most field modules, admin, AI reports
agronomist	Agronomist	Master data, crop cycles, nursery, daily ops, fertigation, pest
farm_supervisor	Farm Supervisor	Nursery, daily ops, irrigation, fertigation, pest, labour, weighing, projects, checkouts, harvest, WhatsApp ops
finance_officer	Finance Officer	Crop cycle budgets, finance ledger, analytics
sales_officer	Sales Officer	Customers, outgrowers, sales orders, payments, logistics
storekeeper	Storekeeper	Inventory, checkouts
quality_officer	Quality Officer	Quality checks
packhouse_supervisor	Packhouse Supervisor	Weighing, harvest, packhouse
driver	Driver	Logistics/dispatch
stable_manager	Stable Manager	Stables (horses, guides, rides)

User seniority is separately ranked (`User::ROLE_RANK`) so that admin actions (edit, deactivate, reset password, view activity) can only be performed on accounts of equal or lower seniority — independent of the module-access map above.

3. Module Map

Modules are grouped below by the three delivery phases used in `routes/web.php`. For each: the controller(s), primary models, and the roles allowed to *write* (all authenticated roles can read unless noted).

Phase 1 — Master Data & Planning

Module	Controller(s)	Key models	Write roles
Expenses	ExpenseController	Expense	All authenticated roles
Master Data (Farms, Blocks, Crops, Assets)	FarmController, BlockController, CropController, AssetController, SetupController	Farm, Block, Crop, Asset	horticulture_manager, agronomist
Land Preparation	LandPreparationController	LandPreparation, LandPreparationTask	daily_ops roles (below)
Crop Planning & Seasonal Budgets	CropCycleController, CropProgramController, CropMonitoringController	CropCycle, CropCycleStage, CropProgram, CropProgramStage, SeasonalBudget, GerminationCheck, PlantPopulationCount, YieldForecast	horticulture_manager, agronomist, finance_officer, md

Phase 2 — Field Operations

Module	Controller(s)	Key models	Write roles
Nursery Management	NurseryBatchController	NurseryBatch, Planting	horticulture_manager, agronomist, farm_supervisor
Daily Farm Operations	DailyActivityController	DailyActivity	horticulture_manager, agronomist, farm_supervisor
WhatsApp Ops (message ingestion)	WhatsappOpsController	WhatsappMessage	md, horticulture_manager, farm_supervisor
Irrigation	IrrigationLogController	IrrigationLog	horticulture_manager, farm_supervisor
Fertigation & Nutrition	FertigationLogController	FertigationLog	horticulture_manager, agronomist, farm_supervisor
Pest & Disease	SprayLogController	SprayLog	horticulture_manager, agronomist, farm_supervisor
Labour & Attendance	LabourAttendanceController	LabourAttendance, Worker	horticulture_manager, farm_supervisor
Weigh-Scale Notifications	WeighScaleReadingController	WeighScaleReading	horticulture_manager, farm_supervisor, packhouse_supervisor
Projects & Task-Splitting	ProjectController, WorkerController	Project, ProjectTask, TaskAssignment, Worker	horticulture_manager, farm_supervisor

Phase 3 — Post-Harvest & Commercial

Module	Controller(s)	Key models	Write roles
Asset Checkouts	AssetCheckoutController	AssetCheckout, AssetStatusHistory	horticulture_manager, storekeeper, farm_supervisor
Inventory & Stores / Procurement	InventoryItemController, ProcurementRequestController	InventoryItem, InventoryTransaction, ProcurementRequest, ProcurementRequestLine	horticulture_manager, storekeeper
Harvest Management	HarvestBatchController	HarvestBatch, HarvestByProduct	horticulture_manager, farm_supervisor, packhouse_supervisor
Packhouse & Traceability	PackhouseLotController	PackhouseLot	horticulture_manager, packhouse_supervisor
Quality Assurance	QualityCheckController	QualityCheck	horticulture_manager, quality_officer
Sales & Customer Contracts	CustomerController, OutgrowerController, SalesOrderController	Customer, Outgrower, SalesOrder, SalesOrderLine	horticulture_manager, sales_officer
Payments & Receipting	PaymentController, MpesaController	Payment, MpesaTransaction	store: sales roles; void: finance roles
Vendors	VendorController	Vendor	md, finance_officer
Logistics & Dispatch	DispatchController	Dispatch	horticulture_manager, sales_officer, driver
Stables (horse rides)	HorseController, GuideController, HorseRideController	Horse, Guide, HorseRide	horticulture_manager, stable_manager
Finance (native ledger)	FinanceController	ChartOfAccount, LedgerEntry, CostAllocation	md, finance_officer
Executive Analytics	AnalyticsController	KpiSnapshot, KpiNarrative	finance_officer, md
AI-Generated Reports	AiReportController	AiReport, AiUsageLog	finance_officer, md, horticulture_manager

Platform / Cross-Cutting

Area	Controller(s)	Notes
Auth	AuthController	Login/register/logout + SMS-OTP email verification and password reset (no email delivery for OTP — SMS only)
Dashboard / Search	DashboardController, SearchController	Landing page and global quick-search
Notifications / Settings	NotificationController, SettingsController	Per-user notification list, profile settings
Administration	UserController, ActivityLogController, InformationSourceController	User onboarding (email allow-listing), role/password management, audit-trail viewer + PDF export; restricted to horticulture_manager (+owner)

4. Core Data Model & Relationships

The domain graph has one long backbone — from land to cash — plus several supporting subsystems hanging off it.

Production backbone

Farm → Block → CropCycle (× Crop) → Planting (× NurseryBatch)
CropCycle → LandPreparation → LandPreparationTask
CropCycle → SeasonalBudget, CropCycleStage (× CropProgramStage)
CropCycle → GerminationCheck, PlantPopulationCount, YieldForecast (in-season monitoring)
CropCycle → FertigationLog, SprayLog, IrrigationLog(via Block), LabourAttendance
CropCycle/Block → HarvestBatch → HarvestByProduct
HarvestBatch → PackhouseLot → QualityCheck
PackhouseLot → SalesOrderLine → SalesOrder → Customer
SalesOrder → Dispatch → Payment → (posts to) LedgerEntry

Supporting subsystems

Subsystem	Core models	Hangs off
Assets & custody	Asset, AssetStatusHistory, AssetCheckout	Farm; linked from IrrigationLog (pumps) and Dispatch (vehicles)
Inventory & procurement	InventoryItem, InventoryTransaction, ProcurementRequest, ProcurementRequestLine	Farm, CropCycle, Project
Labour & projects	Worker, Project, ProjectTask, TaskAssignment, LabourAttendance	Farm, Block, CropCycle
Finance	ChartOfAccount, LedgerEntry, CostAllocation, Expense, Vendor, MpesaTransaction	CropCycle, Block, Project (cost allocation); Expense → Vendor
Analytics / AI	KpiSnapshot, KpiNarrative, AiReport, AiUsageLog	Aggregates across the whole backbone
WhatsApp ingestion	WhatsappMessage	Optionally links to Block / CropCycle once interpreted
Stables (separate business line)	Horse, Guide, HorseRide	Standalone — not connected to the crop backbone
Access & audit	User, ApprovedEmail, ActivityLog	ActivityLog records changes across nearly all models via ActivityObserver

5. Cross-Cutting Flows

WhatsApp message → farm record

Inbound message → `WhatsappMessage` (raw text) → AI/parser interpretation
(`app/Services/Whatsapp/TestMessageParser.php`) proposes a structured record → reviewed in *WhatsApp Ops* UI → approved/rejected by an allowed role → on approval, posted into the relevant operational table (e.g. a `DailyActivity` or log entry).

Payments & M-Pesa

Two payment paths converge on `Payment` : (1) manual receipting by a sales role against a `SalesOrder` , and (2) M-Pesa — either the C2B simulator (demo stand-in for a paybill payment) or, when `MPESA_DRIVER` is switched from `fake` to the Daraja gateway, real Safaricom C2B/B2C webhooks landing on `routes/api.php` and handled by `MpesaWebhookController` . `App\Support\C2BAllocation` and `App\Support\PaymentPosting` contain the allocation/posting rules. Voiding a payment is a finance-only action, separate from recording one.

Finance ledger posting

Cash-affecting events (expenses, payments, disbursements) are posted into a native double-entry-style ledger (`ChartOfAccount` / `LedgerEntry`), with `CostAllocation` attributing spend back to a `CropCycle`, `Block`, or `Project` for margin analysis. `FinanceController::post` is the manual posting entry point.

Traceability (farm to sale)

Every `PackhouseLot` traces back through its `HarvestBatch` to a `CropCycle` / `Block` , and forward through `SalesOrderLine` to the `SalesOrder` and `Customer` it was sold to — exposed via the `/trace` lookup route.

Analytics & AI reporting

`KpiSnapshotService` periodically computes `KpiSnapshot` rows; `GenerateKpiNarrative` (queued job) turns snapshots into plain-language narrative via the Anthropic client. `AiReportController` lets finance/MD/horticulture_manager request ad-hoc AI reports; `FarmContextAssembler` gathers the farm data context fed to the model, and `GenerateAiReport` (queued job) does the generation, logging usage to `AiUsageLog` .

6. External Integrations

Integration	Purpose	Where implemented
Safaricom M-Pesa (Daraja)	C2B customer payments, B2C disbursements to vendors/workers	app/Services/Mpesa (DarajaGateway real, FakeMpesaGateway default via MPESA_DRIVER); webhooks in Api\MpesaWebhookController
Anthropic (Claude)	AI-generated farm reports and KPI narratives	app/Services/Ai/AIClient.php , FarmContextAssembler.php ; config in config/services.php (anthropic.*)
Bonga SMS	OTP delivery for registration/password-reset, and admin-issued password resets	app/Services/SmsService.php ; config services.bonga_sms
Digital weigh scale	External scale device pushes weight readings via bearer-token API	WeighScaleReadingController::ingest , routes/api.php ; token in services.weigh_scale.token
WhatsApp	Field message ingestion (test/simulation mode by default)	WhatsappOpsController , services.whatsapp.test_mode
Slack	Operational notifications	services.slack.notifications
Email (Postmark / Resend / SES)	Configured but registration/reset flows are SMS-first in this build	config/services.php

This document reflects the codebase on branch `brian/develop` as of 2026-07-23. It covers architecture and module boundaries only — it does not enumerate every route, form field, or validation rule. For exact behavior, read the referenced controller/service files directly.